

Desarrollo de aplicaciones de consola en Java

M4: Fundamentos de programación en Java

| **AE4:** Implementar una aplicación básica de consola utilizando las buenas prácticas y convenciones para resolver un problema de baja complejidad acorde al lenguaje Java

Introducción

En esta lección aprenderemos a analizar los estándares de codificación en Java, las convenciones sobre nombres de variables, métodos, clases, paquetes, etc. Así como los diferentes estilos de formato y comentarios. Estas son buenas prácticas que deben aplicarse en cualquier proyecto para mantener un código legible.

Luego veremos cómo crear programas simples por consola en Java. Aprenderemos sobre tipos de datos primitivos y objetos, declaración de variables, operadores aritméticos y lógicos, entrada y salida por consola, estructuras de control de flujo y manipulación de cadenas de texto.

También haremos uso del IDE Eclipse para codificar, compilar y depurar paso a paso la ejecución de nuestros programas Java, identificando y corrigiendo errores y problemas.

Finalmente veremos JavaDoc, una herramienta que nos permite generar documentación en HTML a partir de comentarios especiales insertados directamente en el código fuente.

Aprendizaje esperado

Al finalizar la lección podrás:

Implementar una aplicación básica de consola utilizando las buenas prácticas y convenciones para resolver un problema de baja complejidad acorde al lenguaje Java.

Estándares, convenciones y estilos de codificación

Dentro de Java 8 existen algunas convenciones implícitas que no es obligatorio respetar, pero si lo hacemos, destacaremos por nuestro buen código. Estas convenciones y estándares son los siguientes:

Convención de nombres:

- Los nombres de clases deben comenzar con una letra mayúscula y seguir la convención de nombres en **CamelCase**. Por ejemplo: **MiClase**.

También recordar, que las Clases en Java, NUNCA van en plural.

- Los nombres de métodos y variables deben comenzar con una letra minúscula y seguir la convención de nombres en **camelCase**. Por ejemplo: **miMetodo**, **miVariable**.
- Los nombres de constantes deben estar completamente en mayúsculas, con palabras separadas por guiones bajos. Por ejemplo: **MI_CONSTANTE**.

Organización del código:

- Importa sólo las clases que necesitas en lugar de usar importaciones comodín (`import java.util.*`). Esto hace que el código sea más claro y evita posibles conflictos de nombres.
- Agrupa las importaciones en bloques separados, por ejemplo, las importaciones de paquetes del JDK y las importaciones de paquetes de terceros.

Estilo de sangría y formato:

- Utiliza sangría de 4 espacios para mejorar la legibilidad del código. (No es necesario preocuparse por esto, por qué Eclipse IDE ya lo hace)
- Coloca las llaves de apertura en la misma línea que la declaración (por ejemplo, clase o método), y las llaves de cierre en una línea separada.

Comentarios (Spoiler Alert):

- Utiliza comentarios para explicar el propósito y la lógica detrás de las secciones de código complejas o importantes.
- Documenta tus clases y métodos utilizando JavaDoc, como se mencionó anteriormente.

****No te preocupes que en el último tema de esta lección lo aprenderás en mayor detalle.**

Creando aplicaciones de consola en Java

Una de las posibilidades que nos ofrece Java con su JDK en la versión 25 es que podremos crear aplicaciones ejecutables en nuestras computadoras. Básicamente, podremos crear una aplicación con la cual podremos interactuar mediante la consola.

Las consideraciones que tendremos que tener en cuenta a la hora de hacerlo, son:

1. Tener Java 25 instalado en la computadora.
2. Tener instalado Eclipse IDE
3. Tener alguna aplicación que hayamos programado, mediante la cual interactuamos mediante Scanner(System.in)
4. Y seguir los siguientes pasos para lograrlo.

¿Cómo comienzo?

Tomarás la siguiente aplicación, copiando y pegando en un nuevo proyecto.

```
import java.util.Scanner;

public class Calculadora {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int num1 = 0;
        int num2 = 0;
        int resultado = 0;
        String accionUsuario = "";
        String operador = "";

        System.out.println("Bienvenido a la calculadora de terminal");
```

```

do {
    try {

        System.out.println("Ingrese el primer número:");
        num1 = scanner.nextInt();
        System.out.println("Ingrese el segundo número:");
        num2 = scanner.nextInt();
        System.out.println("Ingrese el operador (+, -, *, /:");
        operador = scanner.next();

        switch (operador) {
            case "+":
                resultado = Calculadora.sumar(num1, num2);
                break;
            case "-":
                resultado = Calculadora.restar(num1, num2);
                break;
            case "*":
                resultado = Calculadora.multiplicar(num1,
num2);
                break;
            case "/":
                resultado = Calculadora.dividir(num1, num2);
                break;
            default:
                System.out.println("Operador inválido");
                break;
        }

        if (operador.equals("+") || operador.equals("-") ||
operador.equals("*") || operador.equals("/")) {
            System.out.println("El resultado es: " +
resultado);
        }

    } catch (Exception e) {
        System.out.println("Error " + e.getMessage());
    } finally {
        System.out.println("-----");
        System.out.println("Stop para detener la app, 's' para

```

```
seguir");

        accionUsuario = scanner.next();

    }
    } while (!accionUsuario.toLowerCase().equals("stop"));
    System.out.println("Calculadora finalizada");
}

public static int sumar(int a, int b) {
    return a + b;
}

public static int restar(int a, int b) {
    return a - b;
}

public static int dividir(int a, int b) throws ArithmeticException {
    return a / b;
}

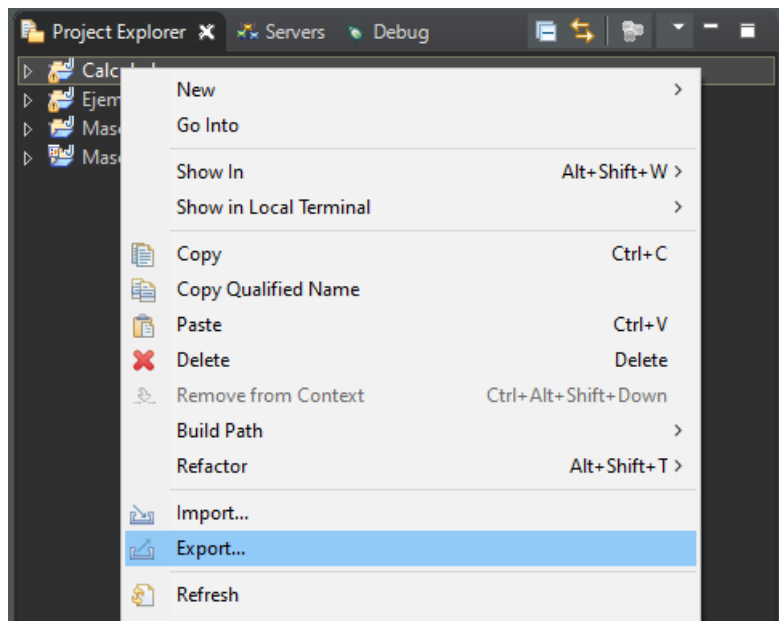
public static int multiplicar(int a, int b) {
    return a * b;
}

}
```

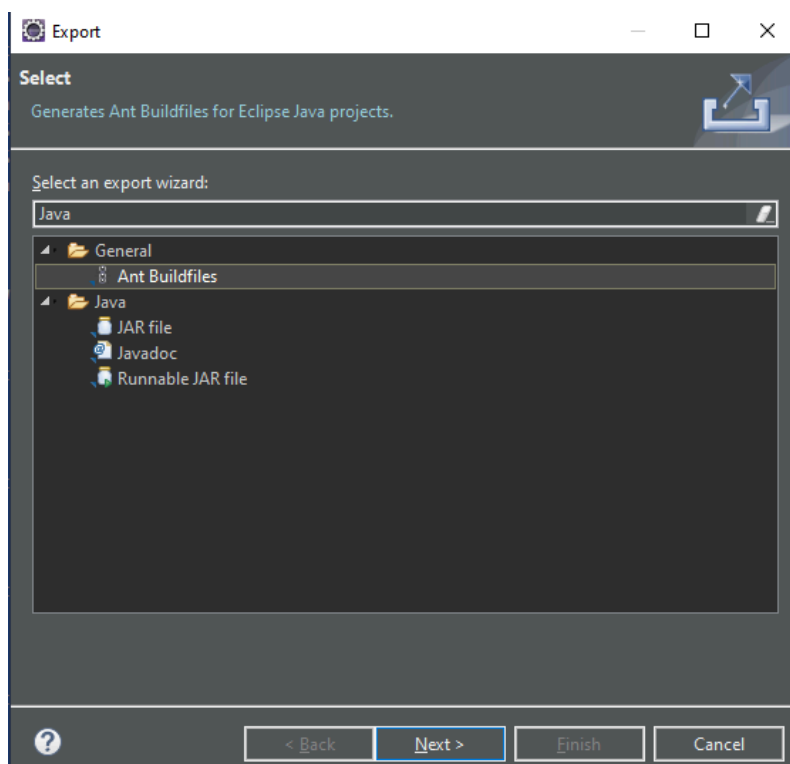
Luego que hayas copiado y pegado, la aplicación y que corra satisfactoriamente en tu IDE, podremos continuar con los siguientes pasos.

Haciendo mi ejecutable .jar

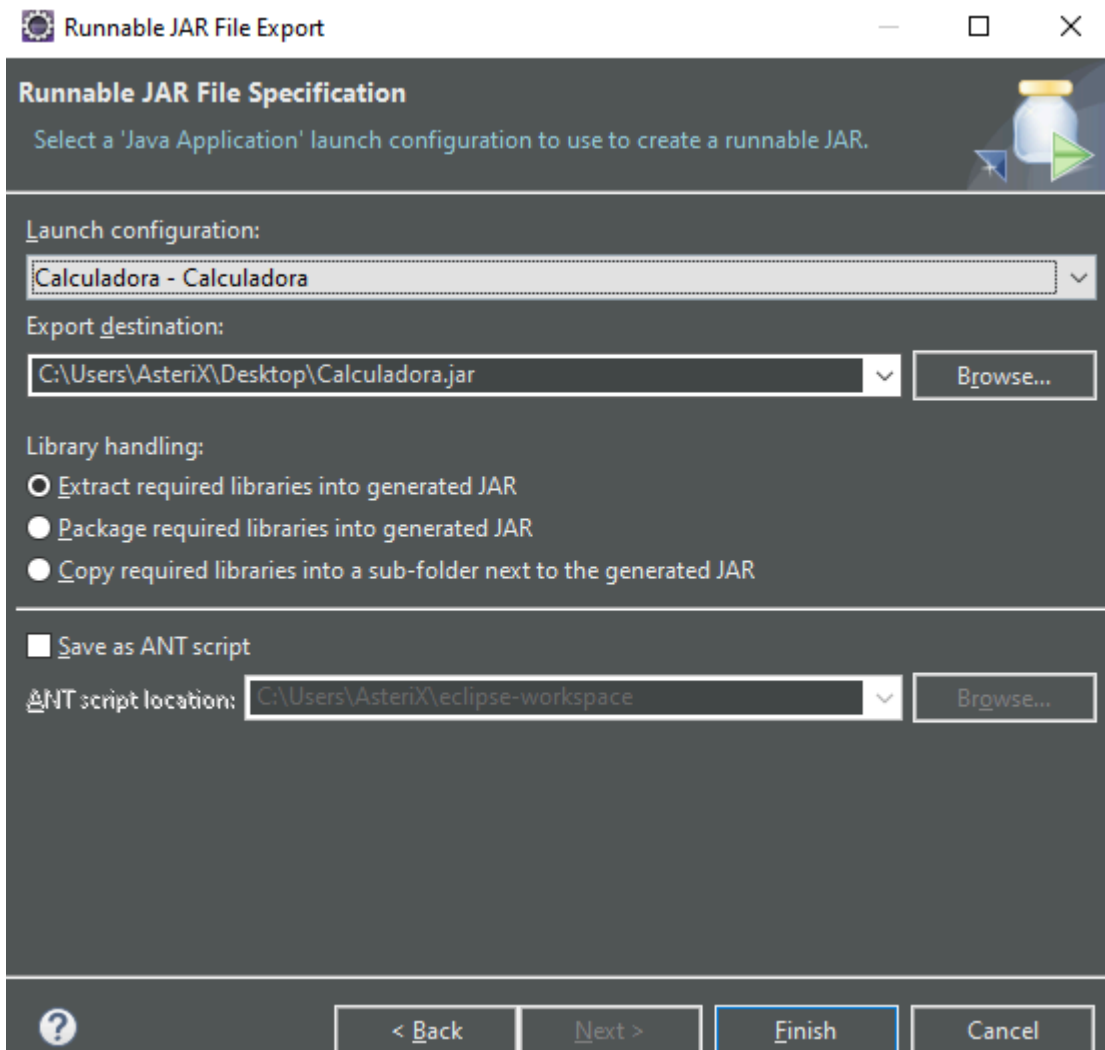
Le damos boton derecho al proyecto y buscamos la opción de “Export”.



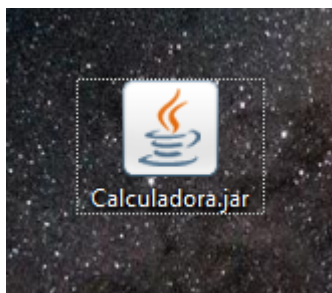
Buscamos la opción de Runnable JAR file.



Seguido a esto, buscamos y seleccionamos la **“Launch configuration”** del proyecto que deseamos generar el ejecutable, seleccionamos el destino donde se generará el ejecutable, en mi caso será el escritorio y finalmente le damos a **“Finalizar”**.



Por último nos generará un archivo como el siguiente.

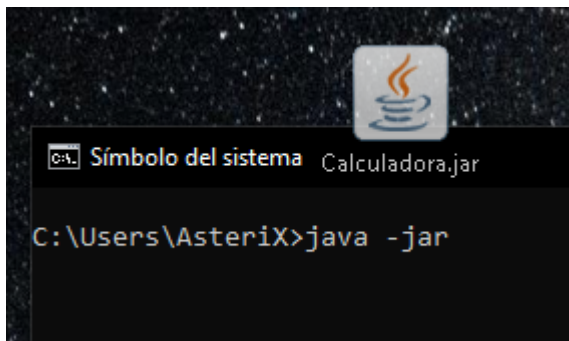


Ejecutando mi archivo desde la terminal

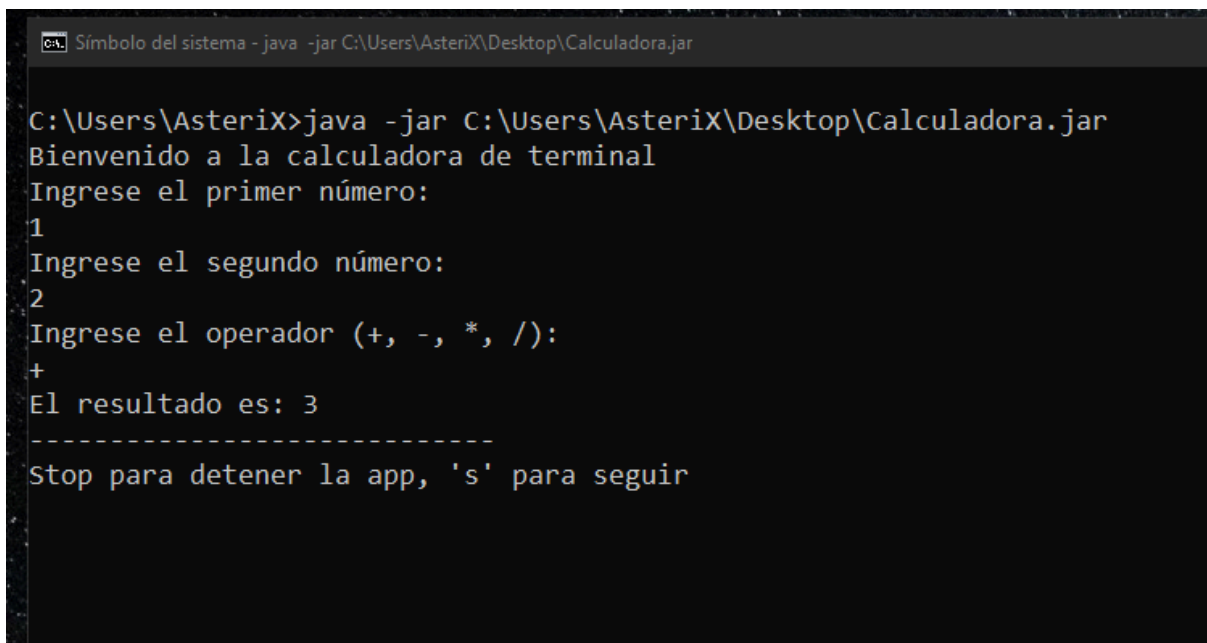
Buscamos en Windows, Mac o Linux nuestra terminal. y en ella escribimos el siguiente comando:

```
java -jar MiApp.jar
```

O también podemos escribir “java -jar ” y arrastramos a la terminal nuestra app, como se muestra en la siguiente imagen



Así, tendremos corriendo nuestra app en la terminal, solo nos faltara interactuar con ella y probarla. En el caso que la deseemos detener solo tendremos que finalizar la app si se pudiese o apretar “Ctrl+c” en la terminal.



Depuración de programas utilizando el IDE

La depuración de código es un aspecto fundamental al programar con Java y es importante por varias razones:

Identificar y corregir errores: La depuración te permite detectar y solucionar errores o bugs en tu código. Mediante la depuración, puedes examinar el estado de las variables, evaluar las condiciones, seguir la ejecución paso a paso y encontrar la causa raíz de los problemas en tu programa.

Ahorrar tiempo y esfuerzo: La depuración eficiente puede ayudarte a ahorrar tiempo y esfuerzo durante el proceso de desarrollo. Al identificar rápidamente los errores y corregirlos, evitas problemas futuros y minimizas el tiempo dedicado a pruebas exhaustivas y revisiones manuales.

Mejorar la calidad del software: La depuración adecuada contribuye a mejorar la calidad general del software que estás desarrollando. Al eliminar los errores y corregir los fallos, asegurar un funcionamiento más confiable y robusto de tu aplicación.

Entender el flujo de ejecución: Al depurar tu código, puedes seguir el flujo de ejecución paso a paso, lo que te permite comprender mejor cómo funciona tu programa. Esto te brinda una visibilidad más clara de cómo interactúan las diferentes partes de tu código y cómo se modifican las variables en cada etapa.

Aprender y crecer como programador: La depuración te brinda una oportunidad invaluable para aprender y crecer como programador. Al enfrentarte a errores y desafíos, desarrollarás habilidades para resolver problemas, mejorar tu capacidad de análisis y ganar experiencia en el dominio del lenguaje y las herramientas de depuración.

La depuración de código es esencial para garantizar la calidad y el rendimiento adecuados de tus aplicaciones Java. Te permite encontrar y solucionar errores, ahorrar tiempo y esfuerzo, mejorar la calidad del software, comprender mejor el flujo de ejecución y fortalecer tus habilidades como programador.

¿Cómo lo hago?

Realizar una depuración de código en Eclipse IDE es una tarea bastante sencilla y te lo mostraremos paso a paso:

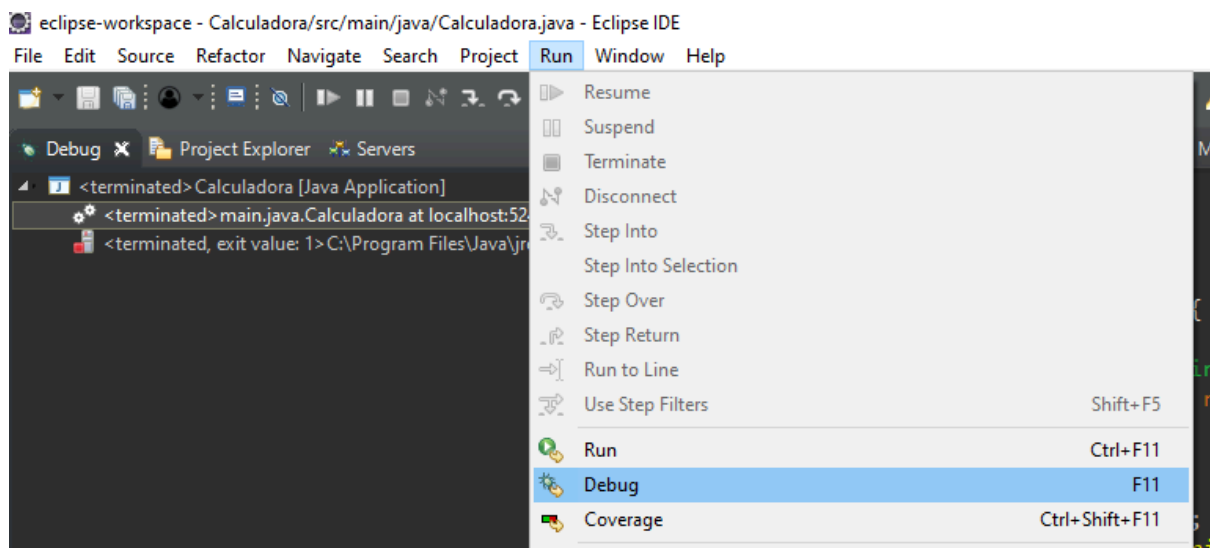
Poner Breakpoints: Los breakpoint son puntos que nosotros determinamos en nuestro código, donde queremos que la ejecución se detenga para poder evaluar estados de variables y procesos, los mismos se colocan clickeando sobre los números a la izquierda de nuestro código

```

6
7
8      public static void
9          Scanner scanne
10
11      int num1 = 0;
12      int num2 = 0;
13      int resultado
14      String accionU
15      String operado
16
17      System.out.pri
18
19      do {
20          try {

```

Depurando código en modo Debug: Para poder depurar nuestro código, debemos anticipar a nuestro IDE que lo haremos, esto se realiza de una forma muy sencilla, luego de haber colocado nuestros Breakpoints, buscaremos en la parte superior de Eclipse la opción Run o ejecutar y haremos click en Debug.



Siendo expertos en Depuración: Una vez que hayamos corrido el modo Debug, se nos abrirá una perspectiva distinta donde tendremos la posibilidad de observar las variables creadas hasta el momento del breakpoint específico, los breakpoint existentes y nos dará la posibilidad de crear expresiones en base a variables.

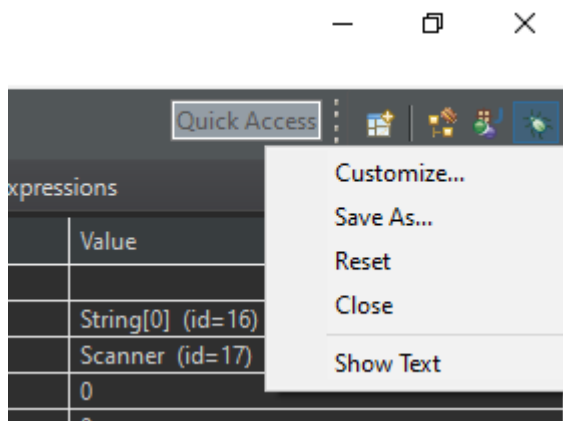
Por último, cuando estemos en un breakpoint, la ejecución se detendrá para poder continuarla deberemos decidir cómo hacerlo, es por eso que les compartimos las posibilidades:

- La tecla F8 nos llevará al siguiente breakpoint, en caso de no existir continuará la ejecución de forma normal.

- La tecla F5 nos llevará al interior del proceso que continua, por ejemplo si en x línea se tiene que ejecutar una función, F5 nos llevará hasta esa función.
- La tecla F6 continuará la ejecución paso a paso, o línea a línea, sin adentrarse en las posibles funciones a llamarse.

No preocuparse, la mejor manera de aprender los atajos de las teclas es poniéndolas en práctica. Lo importante de este apartado, es entender cómo usar la depuración correctamente.

Por último, en el caso que modifiquen la perspectiva de eclipse y no puedan volver a ver las Variables, Breakpoints o Expressions, hacer lo siguiente:



Darle click a “Reset”, y podrán ver la perspectiva del Debug como viene por defecto, también aplica a la perspectiva de Java

Documentando el código con JavaDoc

La documentación en Java se realiza utilizando JavaDoc, una herramienta que genera documentación en formato HTML a partir de comentarios especiales escritos en el código fuente. Los comentarios JavaDoc se agregan encima de declaraciones de clases, métodos, campos y paquetes, y proporcionan información sobre su propósito, funcionamiento y uso.

Debemos recordar que el código bien documentado es un código mucho más legible y mantenible.

¿Cómo se comenta? Los comentarios de documentación se inician con `/**` y se cierran con `*/`. Pero simplemente colocando `/**` o `/*` y dando “Enter”, nos generará la plantilla para documentar.

```
/**
 * public static int sumar(int a, int b) {
 *     return a + b;
 * }
 */
public static int sumar(int a, int b) {
```

```
/**
 *
 * @param a
 * @param b
 * @return
 */
public static int sumar(int a, int b) {
    return a + b;
}
```

A continuación te mostraremos las distintas posibilidades para documentar nuestro código

Documentando una Clase

```
1 package main.java;
2
3 /**
4  * Descripción de la clase.
5  */
6 public class Logica {
7
8     /**
9      * Descripción de la variable
10     */
11     private MiEnum variableEnum;
12
13
14     public Logica() {
15         // TODO Auto-generated constructor stub
16     }
17
18     /**
19      * Descripción de la función,
20      * @param a será una variable entera
21      * @param b será una variable entera
22      * @return Esta función devolverá un valor de la división de a y b
23      * @throws ArithmeticException posible error que lance la función
24      */
25     public static int dividir(int a, int b) throws ArithmeticException {
26         return a / b;
27     }
28 }
29
30
```

Documentando un Enum

```
package main.java;

public enum MiEnum {
    VALOR1, // Descripción del primer valor
    VALOR2  // Descripción del segundo valor
}
```

Documentando una Interfaz

```
2
3 /**
4  * Descripción de la interfaz
5  * @author AsteriX
6  *
7  */
8 public interface MiInterfaz {
9
10     /**
11      *
12      * @param a
13      * @param b
14      */
15     public void func(int a, int b);
16 }
17
```

Cierre

En esta lección pudimos conocer y aprender los distintos estándares a la hora de escribir código en Java, estándares que nos ayudaran a poder destacar como buenos conocedores del lenguaje y poder estar en sintonía con otros desarrolladores.

Por otro lado, aprendimos cómo compilar y generar nuestro archivo ejecutable de Java, aplicación que podremos correr desde cualquier maquina que tenga instalado Java 8

Y finalmente, aprendimos a depurar nuestro código para buscar errores de la forma más profesional posible y cómo documentar nuestra app, con el fin de hacerla más legible y mantenible.

¡Nos vemos de nuevo, más adelante! 

Referencias

- Oracle. (s.f.). Opciones de línea de comandos. Recuperado de <https://www.oracle.com/java/technologies/javase/clopts.html>
- AprenderAProgramar. (s.f.). Documentar con Javadoc. Recuperado de <https://www.aprenderaprogramar.com/documentar-con-javadoc/>
- Adictos al Trabajo. (s.f.). Documentar código Java. Recuperado de <https://adictosaltrabajo.com/2007/05/24/do-tutorial/>
- GeeksforGeeks. (2023, 26 de julio). JAR files in Java. Recuperado de <https://www.geeksforgeeks.org/jar-files-java/>
- Developers.dev. (s.f.). Cómo medir y mejorar la calidad del código Java. Recuperado de <https://developers.dev/articulo/medir-mejorar-calidad-codigo-java>

¡Muchas gracias!

Nos vemos en la próxima lección

